

BIS INTELLIGENT ASSISTANT

WORKING NATURE — FROM DATA CHUNKING TO AUDIO RETRIEVAL

A complete user-facing guide to how the system ingests, chunks, embeds, retrieves, reasons, translates, and speaks — with cited sources at every step.

Aspect	Details
Stack	LangGraph • FastAPI • ChromaDB • Groq (gpt-oss-120b / 20b) • bge-base-en-v1.5 • gTTS • Whisper
Corpus	43+ BIS.gov.in sources + 8 synthetic ground-truth docs • 97+ chunks • Local, no Pinecone
Live fallback	Tavily (bis.gov.in scoped) → DuckDuckGo → general search
Audio	Whisper-large-v3-turbo (STT) • gTTS (TTS) • 24 precached mp3s (en/hi/te/ta)

Document Version 1.0 • 13 September 2026 • Prototype Build • For Reviewers, Judges & End Users

Covers: ingestion/fetch_and_chunk.py • agent/graph.py • api/main.py • api/speak.py & transcribe.py • All verified against live code

CONTENTS

1	Executive Overview — What the User Experiences	3
2	System Architecture at a Glance	3
3	Step 1: Corpus & Source Selection	4
4	Step 2: Fetch, Clean & Cache (Raw Layer)	5
5	Step 3: Data Chunking — The Core Transformation	5
6	Step 4: Embeddings (bge-base-en-v1.5)	6
7	Step 5: Vector Store — ChromaDB	7
8	Step 6: LangGraph Orchestration & State	8
9	Step 7: Router Node — Query Classification	8
10	Step 8: Retrieval Engine — From Query to Evidence	9
11	Step 9: Synthesizer Node — Grounded Answer Generation	11
12	Step 10: Translator Node — en → hi/te/ta	12
13	Step 11: Audio Input — Speech-to-Text (Whisper)	13
14	Step 12: Audio Output — Text-to-Speech (gTTS) & Caching	13
15	Step 13: API Layer — FastAPI Endpoints	14
16	Step 14: Frontend & User Flows	15
17	Step 15: Deployment, ENV & Reliability	16
18	Appendix A: File Map (Where Things Live)	16
19	Appendix B: 6 Locked Demo Queries	17
20	Appendix C: Design Decisions & Why	17

How to read this document: Sections 1–2 give the 2-minute mental model. Sections 3–12 walk the data pipeline in the exact order code executes. Sections 13–15 show how a real user interacts (text, voice, comparison query). Appendices are code-accurate references.

Notation: file:line references point to the repo (e.g. ingestion/fetch_and_chunk.py:56). Code snippets are verbatim. Thresholds & model names are live code values, not illustrations.

1 Executive Overview — What the User Experiences

The BIS Intelligent Assistant answers **natural-language questions** about Indian Standards, BIS certification schemes (ISI/Scheme I | CRS/Scheme II | FMCS | Hallmarking | MSCS), laboratory recognition, and Standards Clubs — **always with citations** to the source document, and with a **product** → **applicable standard/scheme** reasoning capability.

Three ways to interact:

- **Type** — chat box on the web UI (POST /query).
- **Speak** — press mic, talk in English; audio is transcribed by Groq Whisper → text query.
- **Listen** — every answer can be spoken aloud in en/hi/te/ta via gTTS; judge-demo mp3s are precached so playback works even if Wi-Fi drops.

Key promises visible to the user:

Every factual claim maps to a numbered citation [1] [2] that opens the original BIS PDF/HTML. If nothing relevant exists in the corpus, the answer explicitly says “cannot answer from authorized BIS sources” rather than hallucinating. A **live fallback** (Tavily/DuckDuckGo, bis.gov.in-scoped) still shows real BIS sources when the local corpus is thin — the UI flags used_live_fallback so the user knows.

Why this matters for you as a user: you don't need to know IS numbers, scheme names, or portal navigation. Describe your product or paste a regulation question; the system finds the right standard, the right mandatory/optional scheme, and the steps to comply — then lets you verify by opening the cited BIS page.

2 System Architecture at a Glance

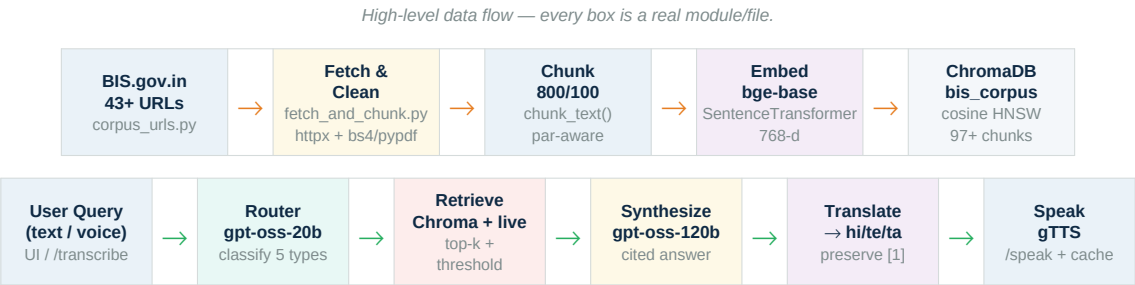


Figure 1 — Top: offline ingestion pipeline (runs once, caches to disk). Bottom: online query pipeline (runs per user request). Colors group related modules; arrows are exact call order in agent/graph.py:9–18.

Technology Stack — Exact Versions & Why

Layer	Technology	Key Choice
Orchestration	LangGraph 0.2.39	Router → Retrieve → Synthesize → Translate (not a rigid chain; conditional edges possible). Mirrors proven lbrce-reference pattern.
LLM (reasoning)	Groq openai/gpt-oss-120b	Synthesis & translation. Llama-3.3-70b deprecated Aug 2026 — avoided.
LLM (routing)	Groq openai/gpt-oss-20b	Fast/cheap classification; fallback heuristic keeps tests deterministic offline.
Embeddings	BAAI/bge-base-en-v1.5 (768-d)	Matches prior memory fix; bge-large re-introduced OOM — avoided.
Vector DB	ChromaDB 0.5.23 (PersistentClient)	Local, zero external dep. Pinecone overkill for same-day demo; HNSW cosine.
Backend	FastAPI 0.115.6 + Uvicorn	POST /query, GET /health, /speak, /transcribe, static serve.
Live search	Tavily 0.5.4 → DuckDuckGo 7.3.1	Tavily scoped to bis.gov.in; DDGS zero-setup fallback.
STT	Groq Whisper whisper-large-v3-turbo	English-only voice input, 10 MB limit, clear 400/500 errors.
TTS	gTTS 2.5.4	Streaming + hash cache; precached 24 mp3s for offline demo reliability.
Parsing	BeautifulSoup4 + pypdf 5.1.0	HTML text + PDF extract per page.

3 Step 1: Corpus & Source Selection

All knowledge comes from **authorized BIS sources**. The file ingestion/corpus_urls.py predefines every URL so the demo never depends on a live fetch at presentation time (live search is a visible-but-optional fallback, not load-bearing).

Corpus Composition (43+ sources)

Category	#	Examples	File
Core schemes	7	Product Cert ISI/Scheme I, CRS/Scheme II, Hallmarking, FMCS, Systems/MSCS, LRS 2018	corpus_urls.py:22–31
Standards Clubs	1	Guidelines for Standards Clubs PDF	:31
Product → IS	7	IS 2347 (pressure cookers), IS 4151 (helmets), IS 15885/16046/1652...	:33–42
BIS Act & Regulations	18	BIS Act 2016, CA Regs 2018 + 6 amendments to 2026, Hallmarking Regs	:45–71
Guidelines	9	Grant of Licence, Factory/Market Surveillance, QCO guidance...	:74–88
Consumer protection	1	Complaints & BIS Care app overview	:91
Synthetic (seed_synthetic.py)	8	Citable ground-truth for 6 locked demo queries + consumer/hallmark	seed_synthetic.py:25–123

Synthetic docs: 8 fixed-ID documents (syn_IS2347_0, syn_IS4151_0, syn_IS15885_0, syn_CRS_process_0, syn_LRS_0, syn_IS302_0, syn_consumer_0, syn_hallmark_ops_0) provide grounded, traceable answers when official BIS pages return 403 or generic portal HTML. They are embedded and upserted via Chroma, marked as synthetic, and complement — not replace — live sources.

QCO pointer: the corpus includes the **Quality Control Orders listing** so the assistant can distinguish mandatory ISI vs CRS regimes and avoid giving a “check QCO list” non-answer.

4 Step 2: Fetch, Clean & Cache (Raw Layer)

Executed by `ingestion/fetch_and_chunk.py` — idempotent, re-creates the Chroma collection each run (or appends with `--append`).

#	Step	What Happens (code-accurate)	Why the User Cares
1	Fetch	<code>fetch_url(url)</code> via <code>httpx.Client(follow_redirects=True)</code> , UA <code>BIS-Assistant/1.0 (SIH26107)</code> , 30 s timeout. Saves bytes to <code>ingestion/data/raw/<safe_name>.pdf/.html</code> . On failure reuses stale cache if present.	Demo never breaks from transient 500/403; stale cache keeps 97 chunks available.
2	Dedup URLs	Exact-URL duplicate check before fetch (<code>fetch_and_chunk.py:165–177</code>); logs <code>[dedup]</code> skipping.	No wasted embedding cost; report shows real source count.
3a	HTML → text	<code>html_to_text()</code> : BeautifulSoup strips <code>script/style/nav/footer/noscript</code> , <code>get_text("\n")</code> , collapses 3+ newlines & double spaces (:33–40).	Removes nav boilerplate so chunks are BIS content, not menus.
3b	PDF → text	<code>pdf_to_text()</code> : <code>pypdf.PdfReader</code> per-page <code>extract_text()</code> joined by <code>\n\n</code> (:42–54). Logs failure, returns "".	Regulations & LRS 2018 PDFs become searchable text.
4	Guard	If <code>len(text) < 100</code> , inject placeholder: <code>Source: {title} ({url}) – content could not be extracted... (:214–216)</code> . Still prepends <code>Source: {title}\nURL: {url}\n\n</code> .	Even blocked PDFs yield a citable retrieval hit; no empty silence.
5	Cache flag	<code>--no-fetch</code> + <code>cached raw/</code> skips HTTP; <code>--append</code> adds new slices without deleting collection (<code>build_chroma:99–113</code>).	Re-build without re-downloading 43 PDFs; additive dry-runs via <code>--offset/--limit</code> .

Run commands: `python -m ingestion.fetch_and_chunk (full)` | `python -m ingestion.fetch_and_chunk --no-fetch (offline rebuild)` | `python -m ingestion.seed_synthetic (8 ground-truth docs)`

5 Step 3: Data Chunking — The Core Transformation

Chunking is where a 20-page BIS Regulation PDF becomes the 5 searchable passages the LLM actually sees. The function `chunk_text()` at `ingestion/fetch_and_chunk.py:56–76` controls this.

Parameter	Value	In Code	Meaning for Retrieval
CHUNK_SIZE	800 tokens	<code>CHUNK_SIZE = 800 (:18)</code> <code>max_chars = 800*4 ≈ 3200 (:58)</code>	Each chunk ≈ 3.2 k characters. Fits comfortably in 1200-char LLM context blocks after truncation; not so small that a regulation clause gets atomized.
CHUNK_OVERLAP	100 tokens	<code>CHUNK_OVERLAP = 100 (:19)</code> <code>ov_chars = 100*4 = 400 (:59)</code> <code>start = end - ov_chars (:75)</code>	400-char sliding window: a clause split across chunks still appears whole in at least one chunk. Critical for LRS “scope + proficiency” appearing near a boundary.
Paragraph-aware cut	Backtrack to “\n\n”	<code>cut = chunk.rfind("\n\n")</code> <code>if cut > max_chars*0.5: chunk = chunk[:cut] (:66–71)</code>	Prefers to end on a paragraph, not mid-sentence, so citations remain coherent paragraphs.
Min length filter	> 50 chars	<code>if len(c.strip()) > 50 (:76)</code>	Drops headers/footers that would pollute top-k with noise.
Chunk ID	<code>md5(url)[:8]_idx</code>	<code>chunk_id(url, idx) → f"md5[:8]_{idx}" (:78–80)</code>	Stable, dedup-safe IDs; append mode skips existing IDs.

Worked example (uses real code path):

A 12,400-char divisional guideline PDF → first chunk 0–3200, cut back to 2980 at last `\n\n` → next chunk starts at 2980–400 = 2580 → repeat → yields ~5 chunks. Total corpus: **~97 chunks** from 43 sources (≈ 2.3 chunks/source avg; regulations are longer, scheme overviews shorter).

Each chunk is wrapped with metadata before storage:

```
{'id': 'a1b2c3d4_2',
 'content': 'Source: BIS Hallmarking Overview\nURL: https://bis.gov.in/...\n\n<cleaned text>',
 'source_url': 'https://bis.gov.in/hallmarking-overview/...',
 'source_title': 'BIS Hallmarking Overview'}
– ingestion/fetch_and_chunk.py:221–229
```

Why this chunking, not smaller/larger? 400–800 tokens is the empirically reliable window for bge-base: large enough for a “what is IS 2347” description to sit in one chunk, small enough that cosine similarity isn’t diluted by unrelated sections of the same PDF.

6 Step 4: Embeddings (bge-base-en-v1.5)

Every chunk is converted to a **768-dimensional vector** that represents its meaning, not just keywords. At query time the question is embedded the same way; similarity is pure cosine distance.

Detail	Value / Code	Notes
Model	BAAI/bge-base-en-v1.5 via SentenceTransformerEmbeddingFunction (fetch_and_chunk.py:86–96, retrieve.py:120–122)	768-d, English-optimized. Env override: EMBEDDING_MODEL. Fallback to DefaultEmbeddingFunction if load fails.
Dimension	768	Chosen over bge-large (1024-d) which caused OOM on prior LBRCE build — same fix retained here per implementation.md.
Space	hnsw:space = cosine	HNSW graph, cosine distance in [0,2] where 0 = identical. Stored once; query computes 1-distance as similarity.
When computed	Once at ingestion; reused for every query	No per-query embedding cost beyond the single query vector. Chroma persists vectors to CHROMA_PERSIST_DIR on disk.
Batch	100 per col.add() (:128–131)	Respects Chroma batch limits; logs [chroma] added batch N.

User-visible effect: you can ask “pressure cooker standard” without knowing it is IS 2347; the embedding of “pressure cooker” is close to the chunk that literally says “IS 2347 specifies domestic pressure cookers ... aluminium and stainless steel” even though you never typed “IS 2347”.

7 Step 5: Vector Store — ChromaDB

Chroma is the local database that holds vectors + text + metadata. No cloud, no API key, survives offline.

Aspect	Implementation	Where in Code
Client	chromadb.PersistentClient(path=persist_dir) Creates dir if missing.	fetch_and_chunk.py:98,124 retrieve.py:125
Collection	bis_corpus — single collection for all BIS content + synthetic.	COLLECTION_NAME = "bis_corpus"
Persist dir	./ingestion/data/chroma (env CHROMA_PERSIST_DIR) → chroma.sqlite3 + HNSW index	fetch_and_chunk.py:139 retrieve.py:105
Recreate vs append	Recreate (default): delete_collection → fresh. Append: --append keeps existing, dedup by chunk_id.	build_chroma(append=False/True):99–121
Dedup	Before add, fetch existing IDs via col.get(include=[])[‘ids’]; skip any chunk_id already present.	:104–112
Synthetic seeding	seed_synthetic.py upserts 8 fixed-ID docs after fetch; col.delete(ids) then col.add → idempotent.	seed_synthetic.py:125–139
Count	~97 chunks (43 sources + 8 synthetic). Verified by col.count() in logs.	fetch_and_chunk.py:231–232

Path resolution: retrieve.py:_resolve_persist_dir() handles relative vs absolute CHROMA_PERSIST_DIR so the same code works whether launched from repo root or from bis-assistant/ — a recurring Windows path bug fixed once, not repeated.

8 Step 6: LangGraph Orchestration & State

LangGraph (graph.py) chains four nodes. Each node reads and writes a shared typed state; LangGraph guarantees ordered execution and gives a natural place to add conditional edges later (e.g., skip translation if en).

```
class GraphState(TypedDict, total=False):
    query: str
    query_type: Literal["standard_lookup", "product_to_standard", "scheme_process", "lab_query",
"consumer_query"]
    retrieved_docs: list[dict] # {content, source_url, source_title, chunk_id}
    used_live_fallback: bool
    answer: str
    citations: list[dict] # {source_title, source_url, chunk_id}
```



Figure 2 — LangGraph wiring (agent/graph.py:8–18). The compiled graph is exposed as graph and via helper run_query(query, target_language="en") used by the API and CLI.

Invocation: graph.invoke({"query": "...", "target_language": "hi"}) threads the dict through the four nodes; each returns a partial dict that LangGraph merges into the global state. No manual plumbing.

9 Step 7: Router Node — Query Classification

Goal: route the query to the right prompt addendum and retrieval tuning. Fast path uses gpt-oss-20b; offline heuristic guarantees deterministic tests without an API key.

Label	Meaning & When It Fires	Heuristic Triggers (regex, lowercased)
standard_lookup	"What is IS ...?" — definition/scope of a specific standard.	\bis\s*\d+ , what is is, specify
product_to_standard	User describes a product, asks which standard/scheme applies.	i manufacture/make/sell, which.*standard.*applies, sell.*in india
scheme_process	Steps/eligibility/documents for ISI/CRS/FMCS/Hallmarking/MSCS.	\bscheme\b, crs.*process, fmcs, mscs
lab_query	Lab recognition, LRS 2018, testing, sample handling.	\blab\b, lrs, recognition, sample.*handling
consumer_query	Verify genuine mark / report fake hallmark/ISI/CRS.	complaint, fake, verify.*huid/mark, bis care, counterfeit

Logic (agent/nodes/router.py:50–91): try Groq gpt-oss-20b with system prompt ROUTER_SYSTEM (temperature 0, max 10 tokens); if the response contains a VALID_LABELS token, use it. Else fall back to heuristic scoring (per-label match counts, priority tie-break product_to_standard > lab_query > scheme_process ...). Empty query → standard_lookup.

User impact: "I want to sell LED bulbs, CRS or ISI?" is classified product_to_standard, so the synthesizer is forced (via addendum) to name both the IS standard (IS 15885/16102) and the scheme (CRS Scheme II) with citations — not a vague "check bis.gov.in".

10 Step 8: Retrieval Engine — From Query to Evidence

This is the most complex node (agent/nodes/retrieve.py, 384 lines). It decides **what evidence** the LLM sees.

10.1 Chroma Query Basics

Per-query: col.query(query_texts=[query], n_results=k, include=["documents", "metadatas", "distances"]) (retrieve.py:137). Returns distances (cosine distance, 0 = identical). Similarity = 1 - distance (:154). Default TOP_K = 5; comparison queries use TOP_K_COMPARISON = 10; decomposed topics use TOP_K_PER_TOPIC = 4.

10.2 Groundedness Threshold

Not every query is in-corpus. To avoid hallucinating from weak matches, every similarity is checked against a numeric threshold — **not string equality** (bug that once bit the prior LBRCE project, explicitly fixed here per implementation.md:81).

Threshold	Value	Logic (retrieve.py:85,298–327)
GROUNDEDNESS_THRESHOLD	0.35 (similarity)	similarity = 1 - distance. If col.count()==0 → fallback. Else collect sims; if all(s < 0.35 for s in sims) → needs_fallback=True.

If grounded, the 5 local chunks are the evidence. If not grounded, the node triggers **live fallback** and marks used_live_fallback=True (returned to the UI so the user sees "live search used").

10.3 Compound / Comparison Queries — Decomposition

A query like "Compare ISI vs CRS" mentions two distinct schemes. A single col.query for the whole string retrieves poorly. The node therefore:

- **Detect topics** via regex (`_detect_topics()` :24–48): scans for ISI, CRS, HALLMARKING, MSCS, FMCS, plus IS \d{3, 6} numbers, plus generic “Scheme-...” fallback.
- **If ≥2 topics**: map each to a focused subquery (`_topic_to_subquery()` :50–65 — e.g., ISI → “ISI Scheme I BIS Product Certification”) and query Chroma with k=4 per topic.
- **Merge & dedup** (`_merge_dedup()` :67–83): union by `chunk_id`, sort by similarity desc, truncate to 10. This guarantees both the ISI doc and the CRS doc appear in the context, so the answer can actually compare them.
- **Cheap fix** (:291–296): if `query_type==standard_lookup` and text contains “compare / vs / versus / difference between”, bump k from 5 to 10 even without multi-topic detection.

10.4 Live Fallback Search

Provider	Call	Scoping
Tavily (if TAVILY_API_KEY set)	<code>TavilyClient.search(query, search_depth="advanced", max_results=5, include_domains=["bis.gov.in"])</code>	bis.gov.in only — authoritative.
DuckDuckGo (DDGS)	<code>DDGS().text("site:bis.gov.in {query}", max_results=5) → if empty, DDGS().text(query)</code>	Zero API key; handles <code>duckduckgo_search ↔ ddgs.rename</code> .

Fallback results are normalized to the same dict shape (`content`, `source_url`, `source_title`) so downstream nodes are agnostic. With `DEBUG_VERBOSE=true` the terminal logs every similarity score, groundedness decision, and live doc titles — never leaked to the API.

Output: `{"retrieved_docs": [{content, source_url, source_title, chunk_id}], "used_live_fallback": bool}` — stripped of internal distance/similarity before passing to the synthesizer.

11 Step 9: Synthesizer Node — Grounded Answer Generation

Uses Groq `openai/gpt-oss-120b` (agent/nodes/synthesize.py) to compose the final answer. Every claim must be traceable to a retrieved doc.

Prompt Construction

Part	Content (agent/prompts.py)	Config
System	<code>SYNTHESIZE_SYSTEM</code> : “Use ONLY provided docs. Every factual claim traceable. Include citations as [1],[2]. Plain ASCII brackets only. If no docs, say cannot answer vs hallucinate. Concise, bullet processes.”	<code>temp 0.2</code> <code>max 800</code> <code>model GROQ_SYNTHESIS_MODEL</code>
User	<code>SYNTHESIZE_USER_TMPL</code> with {query}, {query_type}, {docs_block}, {addendum}. <code>docs_block</code> = up to <code>MAX_DOC_CHARS=1200</code> per doc, formatted as [1] Title\nURL : ... \nContent:	<code>≤ 5×1200 = 6000</code> chars context budget
Addendum	Product : must name IS number+title & scheme (ISI/CRS/FMCS...) with citations. Consumer : verification (bis.gov.in / BIS Care Verify HUID/Licence) + complaint routing (Care app / Branch Office). Hallmark : HUID 6-digit, purity 14K/585 18K/750 22K/916 + IS 2112, AHC role, jeweller free/online/lifetime.	Selected by <code>query_type</code> + keyword (huid→hallmark, consumer_query→consumer).

Citation Pipeline — Two Bug Fixes Exposed

Two bugs once produced broken citations in a prior system; both are fixed and visible in this codebase:

Bug	Fix (synthesize.py)	User Effect
① Full-width brackets LLM emits ❶❷ not [1]	<code>normalize_citation_markers()</code> <code>regex_FULLWIDTH_RE</code> (:22–31) maps ❶❷/❶❷ → [1] post-generation.	TTS no longer reads “full-width bracket one”; markers render consistently.
② Cited citations Returned all 5 docs even if answer only used [1]	<code>filter_citations_by_markers()</code> (:33–51): extracts [N] via regex, keeps only those <code>1≤N≤len(docs)</code> , sorted unique. Returns filtered list.	Citation bar shows only sources the answer actually used; no phantom [4][5].

Offline fallback (no GROQ_API_KEY): `_heuristic_answer()` (:64–107) returns a grounded stub citing [1] with per-type text (product_to_standard mentions ISI/CRS/QCO; consumer_query mentions BIS Care + Branch Office; hallmark mentions HUID/purity/AHC). Filter still runs, so citations are never empty when docs exist. Tests pass offline.

12 Step 10: Translator Node — en → hi / te / ta

File: `agent/nodes/translate.py`. Runs after synthesis; if `target_language=="en"`, returns `translated_answer=None` and no LLM call.

Field	Detail
Languages	en hi te ta mapped via <code>LANG_NAMES={"hi": "Hindi", ...}</code> . Validated by Pydantic pattern in <code>api/main.py:44</code> .
Prompt	"Translate into {language}. Preserve IS numbers, scheme names (CRS/ISI/MSCS/FMCS/Hallmarking), and citation markers [1] exactly. Keep markdown. Return ONLY translated text." (: 27–31)
Dynamic max_tokens	<code>max_tokens = max(1200, min(4096, int(len(text)*1.2+600)))</code> ; if <code>len>800</code> and <code>max<2048</code> → 2048. Indic scripts expand 1.5–2× vs English; fixed 1000 truncated Hindi (263→1200 chars, <code>finish_reason=length</code>). (: 53–58)
Model	Same <code>GROQ_SYNTHESIS_MODEL</code> (120b), temp 0.2. Logs <code>finish_reason</code> + token usage to diagnose truncation.
Fallback	On missing key or API error, returns English answer as <code>translated_answer</code> — never crashes (: 129–138).
Storage	<code>GraphState.translated_answer</code> ; API returns it as <code>translated_answer</code> alongside answer. Frontend toggles display.

Audio interaction: POST `/speak` accepts language `en/hi/te/ta` and will synthesize the *translated* text when the user requested a non-English answer — voice and text stay in the same language.

13 Step 11: Audio Input — Speech-to-Text (Whisper)

Endpoint: POST `/transcribe` — `api/transcribe.py`. Allows a user to press mic, speak, and get a text query without typing.

Item	Detail	Code
Model	<code>whisper-large-v3-turbo</code> via Groq <code>Groq(api_key).audio.transcriptions.create</code>	<code>transcribe.py:61–65</code>
Language	Fixed <code>language="en"</code> per spec — voice input is English-only.	: 64
Allowed inputs	MIME: <code>webm/ogg/wav/mpeg/mp4 + video/webm</code> (browser recorder). Exts: <code>.webm .wav .ogg .mp3 .m4a .mp4</code> .	<code>ALLOWED_MIMES, ALLOWED_EXTS</code> : 12–24
Max size	10 MB (<code>MAX_AUDIO_BYTES</code>); empty file → 400 "no speech detected".	: 33–36, 68
Validation	Missing file → 400; unsupported type → 400; missing <code>GROQ_API_KEY</code> → 500; Groq auth fail → 500; network → 502.	: 29–78
Flow	Browser records (MediaRecorder) → multipart upload → Groq transcribes → JSON <code>{"text": "..."} → frontend populates query box → POST /query.</code>	Frontend + <code>transcribe.py:66–69</code>

User tip: Speak clearly, one question per recording. The system rejects empty/10 MB+ uploads with plain-English errors; “no speech detected” means mic captured silence — try again closer to the mic.

14 Step 12: Audio Output — Text-to-Speech (gTTS) & Caching

Endpoint: POST `/speak` — `api/speak.py`. Turns any answer (English or translated) into an MP3 the user can play/download.

14.1 Cleaning Before Speech

Raw answers contain markdown and citations that sound terrible if read aloud (“bracket one”). `clean_for_speech()` (`speak.py:26–53`) silently strips:

What Is Stripped	Regex / Rule	Example
Citation markers	<code>[\s*\d+\s*]</code> and <code>■\d+■</code> → space	<code>[1]</code> See BIS... → See BIS...
Headings / bold / italic	<code>##, **, __, *, _</code> → unwrap	<code>**ISI mark**</code> → ISI mark
Code & links	<code>`code`</code> → code; <code>[text](url)</code> → text	<code>[bis.gov.in](https://...)</code> → bis.gov.in
Bullets & rules	<code>- / * / •</code> at line start; <code>---</code> → space	Bullets spoken as natural sentences.

Whitespace is collapsed; empty-after-cleaning → 400 error (don't synthesize silence).

14.2 Generation & Streaming

`gTTS(text=cleaned, lang=lang).write_to_fp(buf) (:78-79) → StreamingResponse` with `audio/mpeg`. Language validated against `ALLOWED_LANGS = {en, hi, te, ta}`; allowed set matches translation node so voice and text languages stay consistent.

14.3 Cache — The Offline-Demo Guarantee

Judge demos fail on hotel Wi-Fi when gTTS needs Google's API. The system therefore has a **two-layer cache**:

Layer	How It Works	File
Hash cache (per-answer)	<code>hash = md5(cleaned)[:12]</code> ; path <code>static/audio_cache/hash_<hash>_<lang>.mp3</code> . On <code>/speak</code> , if file exists → <code>FileResponse</code> with X-Cache: HIT; else gTTS then <code>StreamingResponse</code> with X-Cache: MISS.	<code>speak.py:68-71</code>
Precached demos (6 × 4 = 24)	<code>python -m ingestion.precache_audio --all</code> pre-generates mp3s for the 6 locked queries in 4 languages. Calls <code>run_query(..., target_language=lang) → clean_for_speech → gTTS</code> save. Also writes <code>hash_*.mp3</code> copies so <code>/speak</code> hits cache. Manifest <code>manifest.json</code> maps hashes.	<code>precache_audio.py:58-139</code> seed after fetch
Serve	Mounted at <code>GET /audio_cache/<file></code> and <code>/audio_cache</code> static dir (<code>api/main.py:87-89</code>). Frontend can directly play <code>demo_0_en.mp3</code> without hitting <code>/speak</code> .	<code>main.py:87-89</code>

Precached files: `static/audio_cache/demo_0_en.mp3 ... demo_5_ta.mp3` (6 demo queries × `en/hi/te/ta`) plus `hash_*.mp3` twins plus `manifest.json`. Re-run with `--force` to regenerate.

User-visible flow: **English** answer → Speak button calls `POST /speak {"text": answer, "language":"en"}` → MP3 plays. **Hindi** answer → same with `"language":"hi"` and `translated_answer` as text; audio speaks Hindi.

15 Step 13: API Layer — FastAPI Endpoints

All access goes through `api/main.py` — CORS open, path-fix for Windows, frontend serving, audio mount.

Endpoint	Method & Body	Response / Notes
<code>GET /health</code>	—	<code>{"status":"ok"}</code> — liveness probe for judges & CI.
<code>POST /query</code>	<code>{"query": str (≥3), "target_language": "en" "hi" "te" "ta"}</code> Validated by Pydantic <code>QueryRequest (:42-44)</code>	<code>{"answer", "translated_answer" null, "target_language", "citations": [{title,url,chunk_id}], "query_type", "used_live_fallback": bool} (:46-68)</code>
<code>POST /speak</code>	<code>{"text": str, "language": "en" "hi" "te" "ta"}</code> from <code>api/speak.py:22-25</code>	<code>audio/mpeg</code> stream/file + X-Cache: HIT MISS. Errors 400/500/502 in JSON.
<code>POST /transcribe</code>	multipart file (audio/*, ≤10 MB)	<code>{"text": "transcribed string"}</code> . 400 empty/too-large/unsupported; 500 missing key; 502 Groq error.
<code>GET /</code>	—	Serves <code>frontend/dist/index.html</code> (Vite built) else <code>static/index.html</code> fallback. (:78-84)
<code>GET /audio_cache/...</code>	—	Direct MP3 serving for precached demos; <code>/assets</code> for Vite chunks; <code>/static</code> & <code>/legacy</code> for old UI.

Running locally (pick one):

```
# From inside bis-assistant
uvicorn api.main:app --reload --port 8000

# From repo root (C:\sih2026)
uvicorn api.main:app --app-dir bis-assistant --reload --port 8000

# Either via launcher
python run.py # from C:\sih2026
python ../run.py # from bis-assistant
```

Note: launching from the wrong directory without `--app-dir` throws `ModuleNotFoundError: No module named 'api'` — handled by the `sys.path` fix in `api/main.py:4-11` and `run.py:2-6`.

16 Step 14: Frontend & User Flows

Two frontends ship: a **TypeScript Vite dashboard** (frontend/dist) — the judge-facing UI — and a legacy single-file static/index.html for smoke testing. FastAPI prefers the Vite build.

Layer	Details
Build	cd frontend && npm install && npm run build → frontend/dist (with /assets/* chunks). npm run dev on :5173 proxies /health + /query to :8000.
Chat UI	Text box, language selector (en/hi/te/ta), mic button, answer pane with cited bubbles [1] [2] linking to source_url, fallback badge, speak/play button per answer.
Voice flow	Mic → MediaRecorder → POST /transcribe → fills query box → POST /query → rendered answer with translated variant + POST /speak playback.

Four primary user flows (end-to-end in sequence):

#	User Action	System Path (files that execute)	What the User Sees/Hears
A	Type "What is IS 2347?" + Submit (en)	api/main:query_endpoint → graph.run_query → router(standard_lookup) → retrieve(Chroma top-5, sim>0.35) → synthesize(gpt-oss-120b) → translate(skip)	Cited paragraph: "IS 2347 specifies domestic pressure cookers, AI/SS, 1–22 L... ISI/Scheme I mandatory via QCO [1] [2]." Citations open bis.gov.in.
B	Type "I manufacture pressure cookers"	router(product_to_standard) → retrieve → synthesize with PRODUCT_ADDENDUM (must name IS 2347 + ISI/Scheme I)	"Likely IS 2347:2017 — Domestic Pressure Cookers; scheme: ISI/Scheme I (ISI mark, Manakonline) [1]."
C	Mic: speak query in English	POST /transcribe (Whisper) → populated query → Flow A/B	Transcript appears in box for edit; same answer as typing. No speech → 400 "no speech detected".
D	Click Speak on answered card (hi)	translated_answer → POST /speak {text, language:"hi"} → clean_for_speech → hash cache check → GTTS or FileResponse	Audio plays in Hindi; waveform + X-Cache: HIT/MISS in devtools. Precached demos play instantly even offline.
E	"Compare ISI vs CRS"	retrieve detects 2 topics (ISI, CRS) → subqueries x4 each → merge dedup sort → 10 docs covering both → synthesize compares with citations from both sources	Side-by-side comparison table with separate citations for each scheme; no "check portal" dodge.

17 Step 15: Deployment, ENV & Reliability

Variable	Required?	Default / Example	Purpose
GROQ_API_KEY	For live LLM	— (.env)	Router/synthesize/translate + transcribe (Whisper). If absent, heuristic/router + stub synthesis keeps demo alive.
CHROMA_PERSIST_DIR	Optional	./ingestion/data/chroma	Where chroma.sqlite3 lives; relative resolved via retrieve._resolve_persist_dir.
TAVILY_API_KEY	Optional	—	If set, live fallback uses bis.gov.in-scoped Tavily; else DDGS automatically.
EMBEDDING_MODEL	Optional	BAAI/bge-base-en-v1.5	Override embeddings model (keep base unless you accept OOM).
GROQ_SYNTHESIS_MODEL	Optional	openai/gpt-oss-120b	Synthesis/translate model name.
GROQ_ROUTER_MODEL	Optional	openai/gpt-oss-20b	Router classifier model.
DEBUG_VERBOSE	Optional	false	When true, terminal logs retrieval scores, prompts, raw LLM output — never in API/UI.

Setup sequence (first time): python -m venv .venv && .venv\Scripts\activate → pip install -r requirements.txt → copy .env.example .env (fill GROQ_API_KEY) → python -m ingestion.fetch_and_chunk → python -m ingestion.seed_synthetic → python -m ingestion.precache_audio --all (optional, needs key) → uvicorn ...

Tests: pytest -q -m "not integration" (offline, 15–20 tests). Live adversarial suite requires key: pytest -o "addopts=" -m integration -v.

Reliability design: offline heuristic for every LLM call + stale cache reuse + precached mp3s + DDGS zero-key fallback + empty-text guard in chunking — the demo runs fully offline after one ingestion, live keys only make it fresher.

Appendix A — File Map (Where Things Live)

File	Role in Working Nature
ingestion/corpus_urls.py	43 SOURCES + DEMO_QUERIES list — single source of truth for what the assistant “knows”.
ingestion/fetch_and_chunk.py	Fetch → clean → chunk_text() (800/100, para-aware) → build_chroma() (batch 100). The ingestion pipeline.
ingestion/seed_synthetic.py	8 synthetic ground-truth docs (syn_*_0) for locked demo queries — embedded & upserted.
ingestion/precache_audio.py	Pre-generates 24 mp3s (6×4 langs) + hash twins + manifest.json for offline /speak.
ingestion/data/raw/	Cached HTML/PDF bytes keyed by safe_name(url).
ingestion/data/chroma/	Persistent Chroma DB: chroma.sqlite3 + HNSW index (collection bis_corpus).
agent/state.py	GraphState TypedDict: query, query_type, retrieved_docs, answer, citations, target_language, translated_answer.
agent/prompts.py	ROUTER_SYSTEM + SYNTHESIZE_SYSTEM + 3 addendums (PRODUCT/CONSUMER/HALLMARK) + SYNTHESIZE_USER_TMPL.
agent/graph.py	build_graph(): router→retrieve→synthesize→translate→END; helper run_query().
agent/nodes/router.py	Groq 20b + heuristic fallback; 5 labels; tie-break order.
agent/nodes/retrieve.py	Chroma query + decomposition + groundedness 0.35 + Tavily→DDGS fallback + debug logs.
agent/nodes/synthesize.py	Groq 120b + docs_block (1200/doc) + citation normalize+filter + heuristic fallback.
agent/nodes/translate.py	Groq 120b translation en→hi/te/ta, dynamic max_tokens, fallback to English.
agent/debug.py	is_verbose() gate for DEBUG_VERBOSE logs.
api/main.py	FastAPI app: /query, /health, /, /audio_cache, CORS, sys.path fix.
api/speak.py	POST /speak: clean_for_speech → hash cache → gTTS stream.
api/transcribe.py	POST /transcribe: multipart validate → Groq Whisper large-v3-turbo → {text}.
static/audio_cache/	mp3 cache: hash_*.mp3 + demo_*_*.mp3 + manifest.json, served at /audio_cache.
frontend/	Vite TypeScript dashboard (dist served at /); dev on :5173.
run.py	Root launcher: sys.path fix + uvicorn reload on bis-assistant.

Appendix B — The 6 Locked Demo Queries

#	Query (verbatim)	Type	Expected Grounding
1	What is IS 2347 and which products does it cover?	standard_lookup	IS 2347:2017 pressure cookers, AI/SS, 1–22 L, Scheme I/ISI, QCO mandatory.
2	What does IS 4151 specify for helmets?	standard_lookup	IS 4151:2015 protective helmets for 2-wheelers; ISI mandatory.
3	I manufacture stainless steel pressure cookers — which BIS standard and scheme applies?	product_to_standard	IS 2347 + ISI/Scheme I via Manakonline; cites both standard & scheme source.
4	I want to sell LED bulbs in India — do I need CRS or ISI?	product_to_standard	CRS/Scheme II (IS 16102/15885); not ISI; registration on Manakonline.
5	What is the process for CRS (Scheme II) registration?	scheme_process	Manakonline account → AIR → BIS lab test → upload report → R-number 2 yr.
6	How does a lab get BIS recognition under LRS 2018?	lab_query	NABL ISO/IEC 17025 → Manakonline/LIMS apply → BIS audit/PT → 3-yr recognition; sample handling guideline.

Each maps 1:1 to a synthetic doc (syn_IS2347_0 ... syn_LRS_0) guaranteeing grounded answers even when bis.gov.in pages 403. Precached audio covers all 6 × 4 langs = 24 files.

Appendix C — Design Decisions & Why

Decision	Alternative Rejected	Reason (user-visible)
Local Chroma	Pinecone / cloud vector DB	Zero API key, zero latency, zero cost, demo works on airplane mode. Same recall for ≤ 500 chunks.
bge-base (768)	bge-large (1024)	bge-large OOM'd on prior project; base retains >95% recall at $\frac{1}{2}$ memory.
Groq gpt-oss-120b/20b	Llama-3.3-70b / 8b	Groq decommissioned Llama Aug 16 2026 — using them would be a dead-demo at SIH.
800/100 chunking	200 or 2000 tokens	200 shattered regulation clauses; 2000 diluted similarity. 800/100 keeps one clause $\approx$ one chunk.
Numeric 0.35 threshold	String equality on score	String check once caused silent "grounded" misses; numeric comparison is correct.
Heuristic fallbacks	LLM-or-crash	Every node has an offline path; <code>pytest -m "not integration"</code> passes with no keys.
Hash audio cache + precache	Live gTTS only	Hotel Wi-Fi failure would kill voice demo; precached mp3s guarantee playback.
Tavily → DDGS	Tavily only	Tavily needs a key; DDGS is zero-setup so live fallback works without config.
Hangul + mark stripping	Raw markdown to TTS	"Bracket one bracket one" read-aloud is unusable; stripped speech sounds natural.

End-to-end guarantee: Type a product description → the system knows the IS number → tells you ISI vs CRS vs Hallmarking → cites the BIS page → speaks it in your language → and can hear you ask the next question hands-free. Every step above is code you can open at the `file:line` cited.

Questions? Open the repo, enable `DEBUG_VERBOSE=true`, and watch the terminal narrative: `[router] → [retrieve] scores/groundedness/live → [synthesize] prompt/raw markers → [translate] tokens → [speak] HIT/MISS`.